

I'm working on a Python Pygame Centipede game for a Vibe Coding lab.

Task 1: Mushrooms should require 4 hits before being destroyed.

Currently, MUSHROOM_HP is 4, but the mushroom is disappearing after only 3 hits.

Here is the relevant code:

```
Python

MUSHROOM_HP = 4

def hit_mushroom(self, cell):
    self.mushrooms[cell] -= 1

    if self.mushrooms[cell] <= 1:
        del self.mushrooms[cell]

    self.score += 5
```

Please:

1. Identify exactly why the mushroom disappears after 3 hits.
2. Give the smallest possible code change to make it disappear only after the 4th hit.
3. Do not modify any other game functionality.
4. Explain the change briefly.

The issue is this condition:

```
</> Python
```

```
if self.mushrooms[cell] <= 1:
```

Why it disappears after 3 hits

You start with:

```
</> Python
```

MUSHROOM_HP = 4

Each hit does:

Hit	HP after	- = 1	<= 1 ?	Result
1st		3	No	Stays
2nd		2	No	Stays
3rd		1	Yes	Deleted ×
4th		—	—	Never reached

So the mushroom is deleted when its HP reaches 1, rather than when it reaches 0.

Smallest possible change

Change **only** `<= 1` to `<= 0`:

</> Python

```
MUSHROOM_HP = 4

def hit_mushroom(self, cell):
    self.mushrooms[cell] -= 1

    if self.mushrooms[cell] <= 0:
        del self.mushrooms[cell]

    self.score += 5
```

That's it. No other game functionality needs to change.

Now the sequence is:

4 → 3 → 2 → 1 → 0 → destroyed

So the mushroom disappears exactly on the **4th hit**.

Now Task 2

The game contains this function:

```
def mushroom_color(hp):  
    """Return an (r, g, b) colour for a mushroom with  
    pass
```

The function receives the mushroom's remaining HP from 1 to 4.

Implement it so that the mushroom's color changes depending on its remaining HP, making damaged mushrooms visually distinguishable.

Requirements:

- Return valid RGB tuples.
- Full-health mushrooms can use the normal/default appearance.
- Damaged mushrooms should have visibly different colors.
- Do not modify unrelated game logic.
- Keep the implementation simple and appropriate for this Pygame project.

Explain the implementation briefly.

For Task 2, you only need to implement `mushroom_color()` . A simple approach is to use **green for full health**, then progressively change toward **yellow/orange/red** as HP decreases.

</> Python

```
def mushroom_color(hp):  
    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default."""  
    if hp == 4:  
        return None          # Default/full-health appearance  
    elif hp == 3:  
        return (180, 220, 80)  # Light green  
    elif hp == 2:  
        return (255, 200, 50)  # Yellow/orange  
    elif hp == 1:  
        return (255, 80, 50)   # Red
```

What it does

Remaining HP	Color	Meaning
4	None	Full health / normal appearance

Remaining HP	Color	Meaning
3	(180, 220, 80)	Slightly damaged
2	(255, 200, 50)	More damaged
1	(255, 80, 50)	Critically damaged

The important part is that the function returns a valid **(R, G, B) tuple** for damaged mushrooms while returning `None` at full health so the existing/default rendering can remain unchanged.

No other game logic needs to be modified.

Now Task 3.

The game calls:

```
on_segment_hit(segment, self.score)
```

when a centipede segment is shot.

Add a simple visual hit effect, such as a short-lived sparkle or particle effect, when a segment is destroyed.

Requirements:

- Use only Pygame and the existing project.
- No external dependencies.
- Do not rewrite the collision or centipede chain-splitting logic.
- The effect must actually be visible in the running game.
- Modify only what is necessary for this feature.

Please inspect the existing update and draw structure before implementing it.

Yes — for Task 3 I should inspect the existing project structure first, because the correct implementation depends on how your game currently handles `update()` and `draw()`.

Please **upload the Python game file(s)** or paste the relevant parts containing:

- `on_segment_hit(segment, self.score)`
- the main game/class `update()` method

- the main game/class `draw()` method
- the centipede segment class, if it has one
- the main game loop, if separate

I'll then give you the **smallest necessary changes** to add a short-lived Pygame sparkle/particle effect, without touching the collision or chain-splitting logic.

```
import random
import pygame
```

```
CELL, COLS, ROWS = 20, 30, 30
WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4
```

```
def mushroom_color(hp):
    """Return an (r, g, b) colour for a mushroom with the given hit
    points, or None for the default."""
    if hp == 4:
        return None        # Default/full-health appearance
    elif hp == 3:
        return (180, 220, 80) # Light green
    elif hp == 2:
        return (255, 200, 50) # Yellow/orange
    elif hp == 1:
        return (255, 80, 50)  # Red
```

```
def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles,
    sounds, or bonus points here."""
    pass
```

```
def wave_speed_bonus(wave):
    """Return an extra tick-rate multiplier for centipede segments at
    the given wave, or None for the default speed."""
    pass
```

```
class Segment:
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1

    def step(self, mushrooms):
        nxt = self.col + self.dx
        if nxt < 0 or nxt >= COLS or (self.row, nxt) in mushrooms:
            self.row += self.dy
            self.dx = -self.dx
            if self.row >= ROWS - 1:
                self.row, self.dy = ROWS - 1, -1
            elif self.row <= ZONE_TOP and self.dy < 0:
                self.dy = 1
        else:
            self.col = nxt
```

```
class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
        self.mushrooms = {}
        for _ in range(45):
            self.mushrooms[(random.randint(1, ZONE_TOP - 2),
random.randint(0, COLS - 1))] = MUSHROOM_HP
        self.respawn()
        self.spawn_wave()

    def respawn(self):
        self.x = WIDTH / 2
        self.y = HEIGHT - 50
        self.bullet = None
        self.invulnerable = 1.5

    def spawn_wave(self):
        length = 10 + self.wave
```

```
self.chains = [[Segment(0, COLS - 1 - i, -1) for i in
range(length)]]
self.timer = 0.0

def fire(self):
    if self.bullet is None and self.state == "play":
        self.bullet = pygame.Vector2(self.x, self.y - 12)

def hit_mushroom(self, cell):
    self.mushrooms[cell] -= 1
    if self.mushrooms[cell] <= 0:
        del self.mushrooms[cell]
        self.score += 5

def split_chain(self, chain, index):
    segment = chain[index]
    self.chains.remove(chain)
    left, right = chain[:index], chain[index + 1:]
    self.chains.extend(part for part in (left, right) if part)
    self.mushrooms[(segment.row, segment.col)] =
MUSHROOM_HP
    self.score += 100 if index == 0 else 10
    on_segment_hit(segment, self.score)

def update_bullet(self, dt):
    if self.bullet is None:
        return
    self.bullet.y -= BULLET_SPEED * dt
    cell = (int(self.bullet.y // CELL), int(self.bullet.x // CELL))
    if self.bullet.y < 0:
        self.bullet = None
    elif cell in self.mushrooms:
        self.hit_mushroom(cell)
        self.bullet = None
    else:
        for chain in self.chains:
            for index, segment in enumerate(chain):
                if (segment.row, segment.col) == cell:
                    self.split_chain(chain, index)
                    self.bullet = None
```

```

        return

def update(self, dt, keys):
    if self.state != "play":
        return
    self.invulnerable = max(0.0, self.invulnerable - dt)
    self.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) *
PLAYER_SPEED * dt
    self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) *
PLAYER_SPEED * dt
    self.x = max(10, min(WIDTH - 10, self.x))
    self.y = max(ZONE_TOP * CELL + 10, min(ROWS * CELL - 10,
self.y))
    if keys[pygame.K_SPACE]:
        self.fire()
    self.update_bullet(dt)
    self.timer += dt
    effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
    while self.timer >= effective_tick:
        self.timer -= effective_tick
        for chain in self.chains:
            for segment in chain:
                segment.step(self.mushrooms)
        player_cell = (int(self.y // CELL), int(self.x // CELL))
        if self.invulnerable <= 0 and any((s.row, s.col) == player_cell
for c in self.chains for s in c):
            self.lives -= 1
            self.respawn()
            if self.lives <= 0:
                self.state = "lose"
    if not self.chains:
        self.wave += 1
        self.spawn_wave()

def draw(self, screen):
    screen.fill((8, 8, 16))
    for (row, col), hp in self.mushrooms.items():
        color = mushroom_color(hp) or (200 - (MUSHROOM_HP -
hp) * 40, 80, 170)
        center = (col * CELL + CELL // 2, row * CELL + CELL // 2)

```

```

pygame.draw.circle(screen, color, center, CELL // 2 - 1)
pygame.draw.rect(screen, (230, 230, 200), (center[0] - 3,
center[1], 6, CELL // 2 - 1))
for chain in self.chains:
    for index, segment in enumerate(chain):
        center = (segment.col * CELL + CELL // 2, segment.row *
CELL + CELL // 2)
        pygame.draw.circle(screen, (240, 200, 60) if index == 0
else (80, 220, 90), center, CELL // 2)
    if self.bullet:
        pygame.draw.rect(screen, (255, 255, 255), (self.bullet.x - 1,
self.bullet.y - 6, 3, 10))
        if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 ==
0:
            pygame.draw.polygon(screen, (80, 200, 255), [(self.x, self.y -
10), (self.x + 10, self.y + 10), (self.x - 10, self.y + 10)])
            hud = self.font.render(f"Score {self.score} Lives {self.lives}
Wave {self.wave} R = reset", True, (240, 240, 240))
            screen.blit(hud, (10, ROWS * CELL + 6))
            if self.state == "lose":
                label = self.font.render("GAME OVER - Press R", True, (255,
255, 120))
                screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT
// 2)))

```

```

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Centipede")
    clock = pygame.time.Clock()
    game = Game()
    running = True
    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key ==
pygame.K_r:

```

```

        game.reset()
    game.update(dt, pygame.key.get_pressed())
    game.draw(screen)
    pygame.display.flip()
pygame.quit()

```

```

if __name__ == "__main__":
    main()

```

Yep — I inspected the structure. You already have a clean `update()` → `draw()` loop, so we can add the effect with **only three small changes**.

The important detail is that `on_segment_hit()` currently has no access to `Game`, so the simplest approach is to store the particles on the `Game` object and trigger them inside `split_chain()` immediately after the segment is destroyed.

1. Add particles to `reset()`

Add this one line:

</> Python

```
self.particles = []
```

So:

</> Python

```

def reset(self):
    self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
    self.mushrooms = {}
    self.particles = []

```

2. Add the sparkle effect when the segment is destroyed

Replace your current `on_segment_hit()` with:

</> Python

```

def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles, sounds, or bonus points here."""
    pass

```

Actually, because `on_segment_hit()` doesn't have access to `self.particles`, **don't modify this function.**

Instead, add the particles directly in `split_chain()` after `on_segment_hit()`:

</> Python

```
def split_chain(self, chain, index):
    segment = chain[index]
    self.chains.remove(chain)
    left, right = chain[:index], chain[index + 1:]
    self.chains.extend(part for part in (left, right) if part)
    self.mushrooms[(segment.row, segment.col)] = MUSHROOM_HP
    self.score += 100 if index == 0 else 10
    on_segment_hit(segment, self.score)

    center = pygame.Vector2(
        segment.col * CELL + CELL // 2,
        segment.row * CELL + CELL // 2
    )
    for _ in range(8):
        self.particles.append({
            "pos": center.copy(),
            "vel": pygame.Vector2(
                random.uniform(-80, 80),
                random.uniform(-80, 80)
            ),
```

3. Update and draw the particles

At the **start of** `update()`, after the `state` check, add:

</> Python

```
for particle in self.particles[:]:
    particle["pos"] += particle["vel"] * dt
    particle["life"] -= dt
    if particle["life"] <= 0:
        self.particles.remove(particle)
```

So the beginning becomes:

</> Python

```
def update(self, dt, keys):
    if self.state != "play":
        return

    for particle in self.particles[:]:
```

```

        particle["pos"] += particle["vel"] * dt
        particle["life"] -= dt
        if particle["life"] <= 0:
            self.particles.remove(particle)

self.invulnerable = max(0.0, self.invulnerable - dt)

```

Then in `draw()` , add this **before the HUD is drawn**:

</> Python

```

for particle in self.particles:
    pygame.draw.circle(
        screen,
        (255, 220, 80),
        (int(particle["pos"].x), int(particle["pos"].y)),
        2
    )

```

What happens now

When you shoot a centipede segment:

</> Plain text

```

Bullet hits segment
    ↓
split_chain()
    ↓
segment removed
    ↓
8 particles created
    ↓
particles move outward
    ↓
particles disappear after 0.25 sec

```

So you'll get a **short yellow sparkle burst** exactly where the segment was destroyed.

Nothing about collision detection or chain splitting is changed. The only additions are the particle list, particle creation, particle updating, and particle drawing.

Now Task 3.

The game calls:

```
on_segment_hit(segment, self.score)
```

when a centipede segment is shot.

Add a simple visual hit effect, such as a short-lived sparkle or particle effect, when a segment is destroyed.

Requirements:

- Use only Pygame and the existing project.
- No external dependencies.
- Do not rewrite the collision or centipede chain-splitting logic.
- The effect must actually be visible in the running game.
- Modify only what is necessary for this feature.

Please inspect the existing update and draw structure before implementing it.

I inspected your structure. Since `split_chain()` already calls `on_segment_hit(segment, self.score)`, the cleanest minimal solution is to make `on_segment_hit()` create a small global particle list, then update/draw that list through the existing game loop.

However, because your current `on_segment_hit()` is a standalone function and has no access to `Game`, **the smallest safe change is to pass the particle list to it**—but that would change the existing call.

So, to preserve the exact call:

```
</> Python
```

```
on_segment_hit(segment, self.score)
```

use a small module-level particle list.

1. Add this near the constants

```
</> Python
```

```
particles = []
```

2. Implement `on_segment_hit()`

Replace `pass` with:

</> Python

```
def on_segment_hit(segment, score):
    """Create a short-lived sparkle effect when a segment is destroyed."""
    center = pygame.Vector2(
        segment.col * CELL + CELL // 2,
        segment.row * CELL + CELL // 2
    )

    for _ in range(8):
        particles.append({
            "pos": center.copy(),
            "vel": pygame.Vector2(
                random.uniform(-80, 80),
                random.uniform(-80, 80)
            ),
            "life": 0.25
        })
```

3. Update the particles

At the beginning of `Game.update()` , after:

</> Python

```
if self.state != "play":
    return
```

add:

</> Python

```
for particle in particles[:]:
    particle["pos"] += particle["vel"] * dt
    particle["life"] -= dt

    if particle["life"] <= 0:
        particles.remove(particle)
```

So:

</> Python

```
def update(self, dt, keys):
    if self.state != "play":
        return

    for particle in particles[:]:
```

```

        particle["pos"] += particle["vel"] * dt
        particle["life"] -= dt

        if particle["life"] <= 0:
            particles.remove(particle)

    self.invulnerable = max(0.0, self.invulnerable - dt)
    ...

```

4. Draw the particles

In `Game.draw()` , after drawing the centipede and before the HUD, add:

</> Python

```

for particle in particles:
    pygame.draw.circle(
        screen,
        (255, 220, 80),
        (int(particle["pos"].x), int(particle["pos"].y)),
        2
    )

```

Result

When a segment is destroyed:

</> Plain text

```

Segment shot
↓
split_chain()
↓
on_segment_hit(segment, score)
↓
8 yellow particles appear
↓
Particles move outward
↓
Disappear after 0.25 seconds

```

This leaves your existing:

</> Python

```

on_segment_hit(segment, self.score)

```

call **unchanged**, and does not touch the collision or chain-splitting logic.

One small note: because `particles` is module-level, add `particles.clear()` inside `Game.reset()` if you want a reset to immediately remove any currently visible particles:

</> Python

```
particles.clear()
```

That's optional but appropriate for keeping reset behavior clean.

```
for particle in particles:
    pygame.draw.circle(
        screen,
        (255, 220, 80),
        (int(particle["pos"].x), int(particle["pos"].y)),
        2
    )
where?
```

Put it inside `Game.draw()`, after the centipede is drawn and before the bullet/player/HUD section.

In your existing `draw()` function, find this:

</> Python

```
for chain in self.chains:
    for index, segment in enumerate(chain):
        center = (segment.col * CELL + CELL // 2, segment.row * CELL + CELL // 2)
        pygame.draw.circle(screen, (240, 200, 60) if index == 0 else (80, 220, 90), center, CELL // 2)
```

Immediately after it, add:

</> Python

```
for particle in particles:
    pygame.draw.circle(
        screen,
        (255, 220, 80),
        (int(particle["pos"].x), int(particle["pos"].y)),
        2
    )
```

So that section of your `draw()` should look like:

</> Python

```

for chain in self.chains:
    for index, segment in enumerate(chain):
        center = (segment.col * CELL + CELL // 2, segment.row * CELL + CELL // 2)
        pygame.draw.circle(
            screen,
            (240, 200, 60) if index == 0 else (80, 220, 90),
            center,
            CELL // 2
        )

# Task 3: draw hit particles
for particle in particles:
    pygame.draw.circle(
        screen,
        (255, 220, 80),
        (int(particle["pos"].x), int(particle["pos"].y)),
        2
    )

if self.bullet:

```

That's the correct location. The particles will be drawn **on top of the game area**, before the HUD.

Now implement Task 4 only.

The game uses:

```
effective_tick = TICK / (wave_speed_bonus(self.wave))
```

Implement:

```
def wave_speed_bonus(wave):
```

so later waves make the centipede progressively faster.

Use this intended formula:

```
1 + 0.15 * (wave - 1)
```

Therefore:

- Wave 1 = 1.00x
- Wave 2 = 1.15x
- Wave 3 = 1.30x
- Wave 4 = 1.45x

Do not modify unrelated game logic.

Replace only the `pass` in `wave_speed_bonus()` with the intended formula:

`</>` Python

```
def wave_speed_bonus(wave):  
    """Return an extra tick-rate multiplier for centipede segments at the given wave, or None for the default  
    return 1 + 0.15 * (wave - 1)
```

Result

Wave	Speed multiplier
1	1.00x
2	1.15x
3	1.30x
4	1.45x

You do not need to change:

`</>` Python

```
effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
```

Because a larger multiplier makes `effective_tick` smaller, the centipede moves more frequently and therefore becomes faster on later waves.